

Comprehensive Image Segmentation Benchmarking

Samuel Collins · Ph.D. Student, Department of Cyber-Physical Systems · Clark Atlanta University
samuel.collins@students.cau.edu

Computer Vision - Image Segmentation Benchmarking · Instructor: Dr. Kishor Datta Gupta · 2026-09-22

Repository: https://github.com/srcollins785/Samuel_Collins_Segmentation_Benchmark

Summary

The benchmark has not produced results yet.

Ten segmentation methods were trained and evaluated on one shared COCO 2017 subset - 5,000 training, 1,000 validation and 1,000 test images over six classes - under a common protocol of 25 epochs, AdamW at 1e-4, batch size 8, at 256x256 resolution, seed 42. Split checksum `a61d376d4a98c607`.

Every table and figure in this report was generated from saved evaluation outputs by `run_benchmark.py --tables-only`. No number here was typed by hand.

The four tasks, and which models do which

Task	Required output	Example
Image classification	One label for the image	"Dog" describes the whole photograph
Object detection	Class and box per object	Dog: [x1, y1, x2, y2]
Semantic segmentation	A class label for every valid pixel	All cars share one label
Instance segmentation	Class and separate mask per object	Person #1 and Person #2 have distinct masks

Seven models here produce semantic segmentation (FCN, U-Net, U-Net++, SegNet, DeepLabV3, PSPNet, SegFormer) and need not distinguish two objects of the same class. Two produce instance segmentation (Mask R-CNN, YOLO-seg) and must keep separate masks for objects that share a label. One is a non-learned baseline (K-Means).

R-CNN is not Mask R-CNN

The family evolved in four steps, and only the last one segments:

- **R-CNN (2014)** ran selective search for region proposals, warped each to a fixed size and pushed every one through a CNN separately. Thousands of forward passes per image.
- **Fast R-CNN (2015)** ran the CNN once over the whole image and pooled features per proposal instead, collapsing the cost.
- **Faster R-CNN (2015)** replaced selective search with a Region Proposal Network, making proposals learned and the pipeline end-to-end.

- **Mask R-CNN (2017)** added a mask branch per region of interest and replaced RoI Pool with RoI Align.

Only the last predicts masks. A detection-only model does not satisfy an instance-segmentation requirement, which is why the instance track here uses Mask R-CNN and a YOLO **segmentation** variant rather than their detection counterparts.

Dataset

A COCO 2017 subset over background plus five foreground classes: person, car, bicycle, dog and cat. COCO was chosen by the assignment because it supplies polygon and RLE masks, multiple instances per image, and the same annotations serve both tracks - instance annotations become class-level masks for the semantic models and stay separate for the instance models.

Splits

Generated at **seed 42**, checksum `a61d376d4a98c607`. Every model records this checksum with its results; two models whose checksums differ were not evaluated on the same test set and their comparison would not be one.

Split	Images	Instances
train	5,000	21,410
val	1,000	4,111
test	1,000	4,352

All three splits are drawn from `train2017`. COCO's own layout invites taking the test split from `val2017`, but that would make the test set a different annotation pass, and any validation-to-test gap would then be partly a dataset difference rather than a generalization gap. Drawing all 7,000 images from one pool makes the three partitions identically distributed by construction - visible in the class shares below, which agree to within a third of a percentage point.

Class balance, and why it shapes every result

Split	background	person	car	bicycle	dog	cat	ignored
train	83.00%	13.91%	1.03%	0.26%	0.77%	1.03%	0.38%
val	82.89%	13.94%	0.91%	0.18%	1.08%	1.00%	0.58%
test	82.73%	14.16%	1.15%	0.27%	0.55%	1.14%	0.46%

Background is 83% of labeled pixels and bicycle is 0.26% - a factor of more than 300. Two consequences run through everything below. First, a model that predicts background everywhere scores about 0.83 pixel accuracy, which is why Section 22 asks for mIoU and why the K-Means baseline's result is instructive rather than embarrassing. Second, six-class mIoU is steered by background and person, so the per-class table is reported before the mean rather than after it.

Mask creation policies

Section 8 requires these documented and applied consistently across every model. They are defined once, in `_config.py`, and both tracks read them from there:

- **Overlapping annotations** - smaller-instance-wins (paint in descending area order). The alternative, first-annotation-wins, would let a large object erase a small one sitting on it, and small objects are already the hardest case.
- **Crowd regions** - `iscrowd=1` regions marked `IGNORE_INDEX`; excluded from instance targets. A crowd blob marks real objects of a real class that COCO declined to annotate individually, so calling it background would teach every model that crowds are background. It is also not a usable instance. It is ignored in both, and retained in the evaluation annotation file where COCOeval uses it to avoid penalizing a detection that lands on an unannotated group.
- **Non-selected categories** - non-selected COCO categories are background.
- **Eligibility** - at least one non-crowd selected-class annotation, minimum 1 foreground pixel(s).

Masks are 8-bit PNGs at native resolution, with 255 as the ignore sentinel. PNG because it is lossless: a JPEG mask would interpolate class IDs into each other at every boundary and invent classes that were never annotated.

Models

Section 10 notes that an architecture name is not a configuration. Each model's backbone, initialization source and measured size are recorded with its results and reproduced here.

Model	Task	Backbone	Initialization
K-Means	traditional	none	none (unsupervised, fitted per image)

Initialization is not uniform, and that confounds the architecture comparison. FCN and DeepLabV3 load COCO segmentation weights including a trained head. PSPNet and SegNet load ImageNet backbones. SegFormer loads a pretrained encoder with a randomly initialized decode head. U-Net and U-Net++ are entirely from scratch. This is a property of what pretrained weights exist, not a choice, but it means a ranking on mIoU is partly a ranking on how much each model was given to start with.

The traditional baseline

Section 10 is explicit that K-Means cluster IDs are not class labels, and forbids matching clusters to test labels to inflate a score. The mapping here is fitted on training images only and frozen: clusters are described by a coarse, quantized descriptor of color, position and coverage, and each descriptor bin takes the class that won it across the training set. `predict()` takes an image and has no parameter through which a mask could be passed, so the inflation the assignment warns about is unavailable rather than merely discouraged.

The result is worth reading carefully. K-Means scored **0.825 pixel accuracy** and **0.139 mIoU**, but only **0.001 foreground mIoU**. Of 58 descriptor bins, 53 map to background and 5 to person; no bin maps to car, bicycle, dog or cat at all.

That is close to the degenerate solution - predict background almost everywhere - and the pixel accuracy of 0.825 is almost exactly the background share of the test set. It is not a bug and it is not a tuning failure to be fixed. It is what an honest color clustering produces on an 83%-background dataset: majority vote over any descriptor bin lands on background, because background is what most pixels are. A mapping fitted against test labels would have scored far higher and measured nothing.

It also demonstrates the point Section 22 makes about metric choice more cleanly than any trained model could. Pixel accuracy of 0.825 sounds like a working segmenter. Foreground mIoU of 0.001 is the same model, described honestly.

What K-Means actually does is group pixels that look alike. That is a different task from naming objects: it separates a dark dog from a light lawn, but it also separates the dog's sunlit flank from its shadowed one, and merges a gray cat with gray pavement. Those are properties of color clustering, not deficiencies to be tuned away.

Experimental protocol

Setting	Value
Input resolution	256x256
Epochs	0
Batch size	1
Optimizer	AdamW
Learning rate	1e-4
Weight decay	1e-4
Seed	42
Mixed precision	not used (float32 throughout)

Resolution is a documented deviation. Section 13 specifies 512x512 and permits 256x256 where compute is limited, applied consistently. It is applied consistently here, and the decision was made from measurement rather than convenience: training the seven semantic models for 25 epochs was measured at roughly 9 hours at 256 and 35 hours at 512 on this hardware, and the full ten-model benchmark at 512 would have exceeded two days of continuous compute.

Preprocessing and augmentation

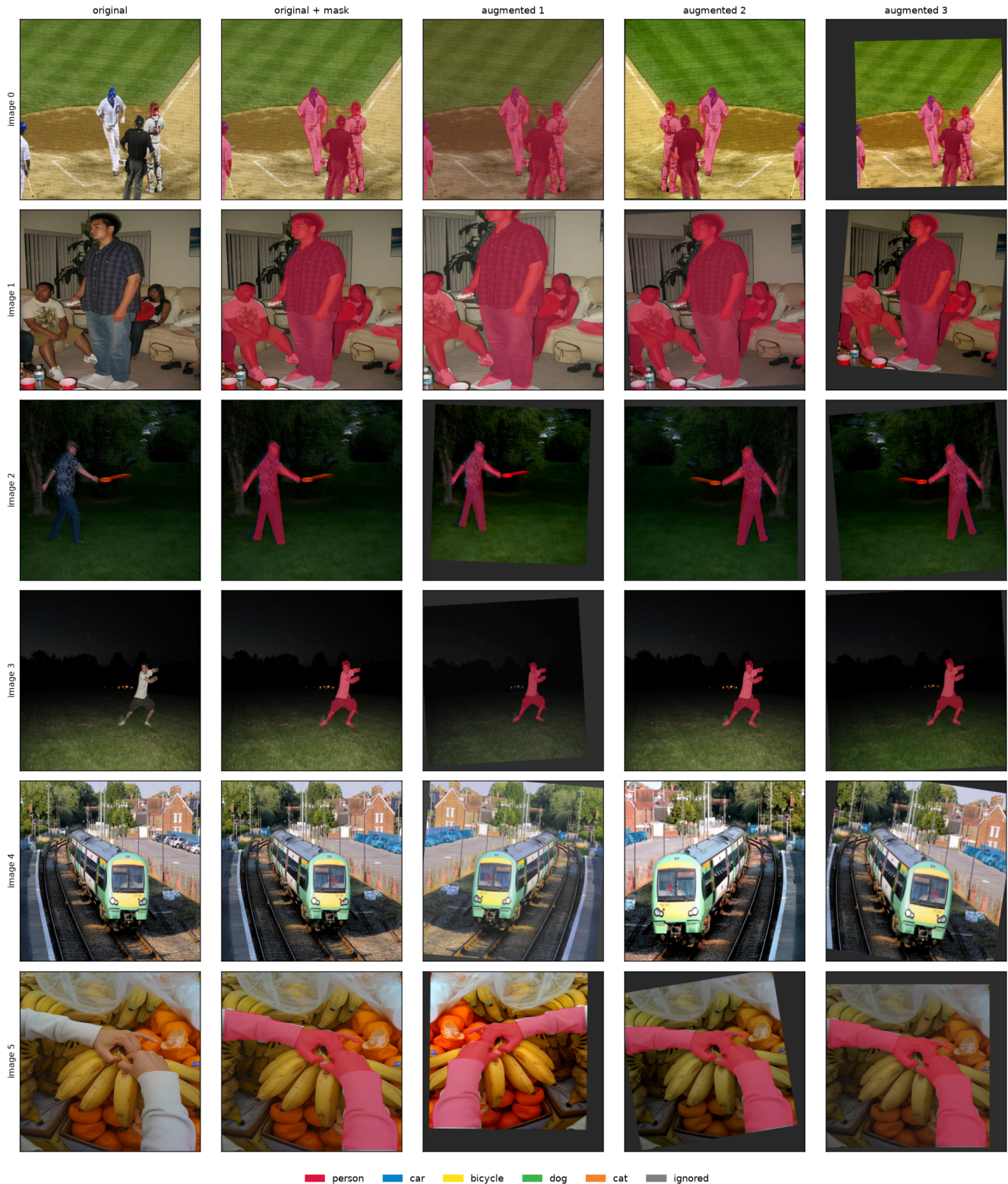
Training applies, in this order:

Validation and test apply deterministic resizing and normalization only, so a validation score changes between epochs because the weights changed and for no other reason.

Every geometric transform rewrites the image, the semantic mask, each instance mask and each box together. Section 15 warns that a flipped image paired with an unchanged mask is invalid training data, and asks for transformed pairs to be visualized before benchmarking.

`scripts/verify_transforms.py` does that and checks it numerically first, because a figure only catches a desync large enough to see and a three-pixel drift would look fine at figure resolution while making the boundary results meaningless.

Section 15 verification: geometry applied to image and mask together



Section 15 verification: geometry applied to image and mask together

Two decisions inside that pipeline are easy to get wrong. Boxes are re-derived from the transformed mask rather than transformed directly, because rotating a box's corners and taking the extent gives a box strictly larger than the object. And pixels a transform invents - rotation corners, crop padding - are marked

ignore rather than background, so they never enter the loss as an observation.

Loss

Instance models keep their own losses, as Section 17 requires. Mask R-CNN trains on classification, box regression, mask and the two RPN terms; YOLO uses ultralytics' box, class, DFL and mask losses. Neither borrows the semantic recipe: they are trained against object proposals rather than a dense label map, and forcing a per-pixel cross-entropy onto them would be a different task, not a harder version of the same one.

Metrics and conventions

Section 22 requires four conventions declared and held to throughout. They are defined once in `metrics.py` and recorded into every result file, so a CSV read later carries the convention that produced it.

- **Which pixels count.** Pixels marked ignore - crowd regions, and canvas that augmentation invented - are excluded before the confusion matrix is accumulated. They enter neither numerator nor denominator of anything.
- **Background.** Reported both ways. `mIoU` covers all six classes and is the figure the required table uses; `mIoU (fg)` covers the five object classes. Reporting only the first flatters every model, since background is 83% of pixels and trivially easy.
- **Absent classes.** A class with no ground-truth and no predicted pixels has an undefined IoU and is excluded from the mean, not scored zero or one. Scoring it zero would punish a model for a class the test set never showed it.
- **Averaging.** Macro: per class, then averaged. Dice and pixel F1 are the same quantity on the same counts, so one function computes both and they cannot drift apart.

Metrics are accumulated as counts across the whole test set and computed once at the end, not averaged over per-batch scores. Those are different numbers: a per-batch average weights a batch holding four bicycle pixels as heavily as one holding forty thousand.

Keeping the two tracks apart

Section 26 forbids comparing semantic mIoU with instance mask AP as though they measured the same thing, and this report does not. The two quality tables are separate, no figure places them on a shared axis, and the only combined table is efficiency - which is legitimately comparable, because every model takes an image and takes time to do it.

Semantic segmentation results

Every number in this section comes from `semantic_segmentation_results.csv`, which the benchmark generates from the per-model result files. All models were evaluated on the same 1,000 held-out test images, and no model saw them during training or checkpoint selection.

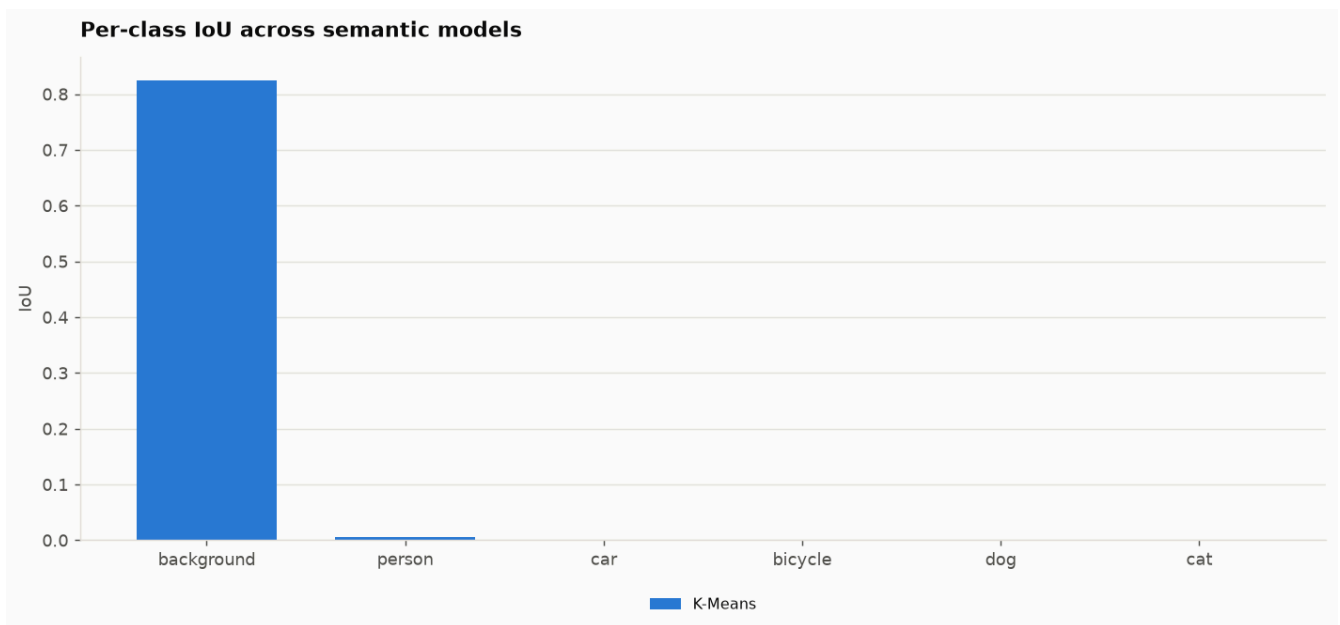
Model	Pixel acc.	mIoU	mIoU (fg)	Dice	Precision	Recall
K-Means	0.825	0.139	0.001	0.153	0.525	0.167

The two mean-IoU columns differ by 0.137 on average, and the gap is not noise. Background is 83% of all valid pixels in this dataset and is trivially easy, so including it lifts every model's mean. The six-class figure is the one the required table reports; the foreground figure is the one that describes how well the five classes anyone cares about were actually segmented. Both are given here so neither can be read as the other.

Per-class IoU

Section 23 asks for one column per semantic model. The dashes in the required format mean a value to be measured; here an N/A means the class was absent from both the prediction and the ground truth for that model, which is not the same as a score of zero and is not averaged as one.

class	kmeans
background	0.825
person	0.006
car	0.000
bicycle	0.000
dog	0.000
cat	0.000



Per-class IoU across semantic models

Boundary quality

Boundary precision, recall and F1 follow Csurka et al. (2013), at a tolerance of **3 pixels** and an evaluation resolution of **256x256**. A predicted boundary pixel counts as matched when a ground-truth boundary pixel lies within that distance, and vice versa for recall.

This is reported separately from IoU because IoU cannot see it. A person occupying 40,000 pixels whose outline is wrong by three pixels all the way around loses under 4% of its IoU while looking visibly wrong everywhere it matters. Two models can tie on mIoU and differ substantially here.

Model	B-Precision	B-Recall	Boundary F1	mIoU (for reference)
K-Means	0.520	0.105	0.126	0.139



Boundary F1 at a 3-pixel tolerance

Instance segmentation results

No instance results have been generated yet.

Efficiency

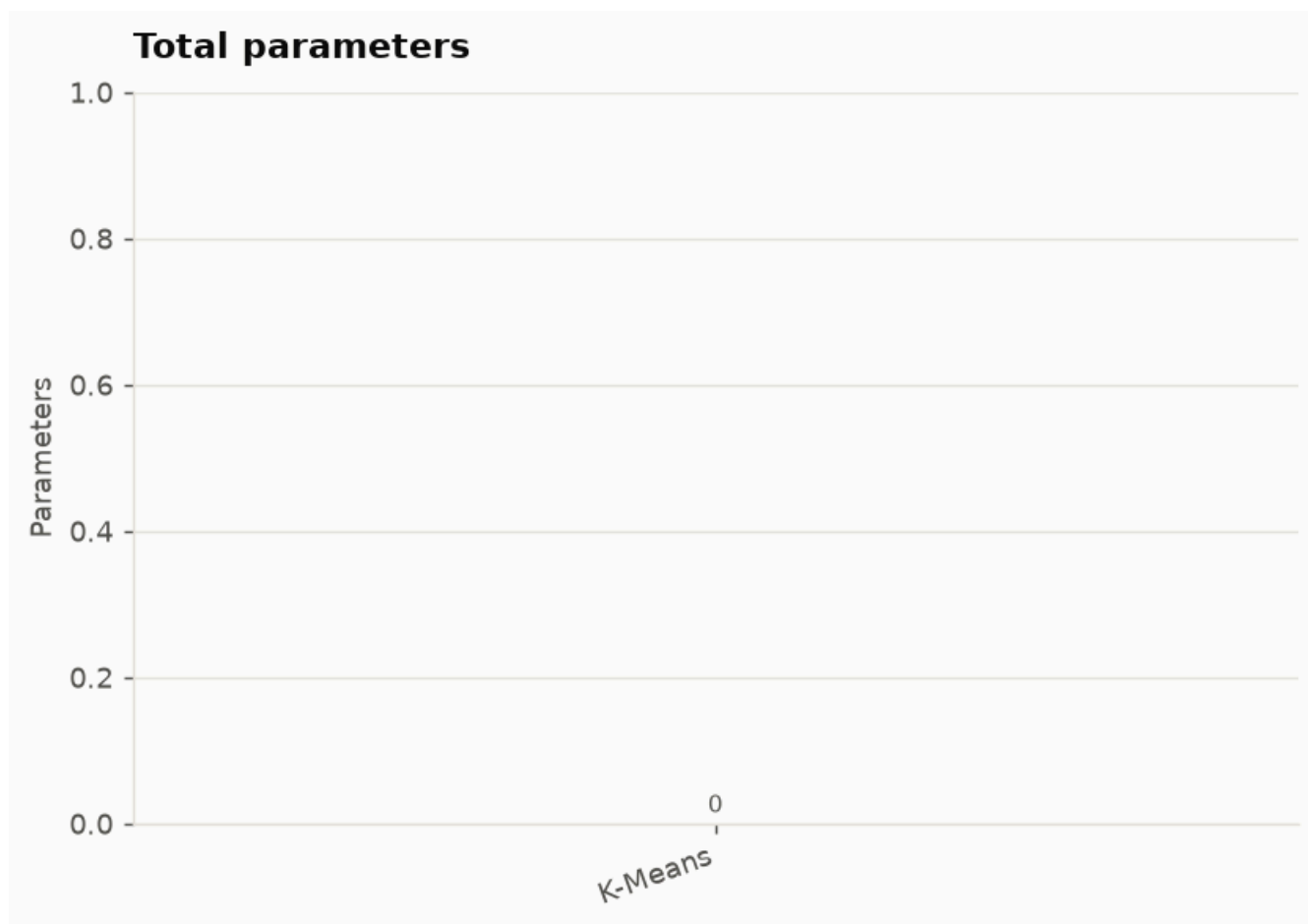
Efficiency is the one axis on which the traditional baseline, the semantic networks and the instance networks are directly comparable: they all take an image and all take time to do it. Quality is not comparable across tracks, which is why those tables are kept apart and this one is not.

All measurements were taken on Apple M4 Max using the mps backend at float32 precision, torch 2.14.0, seed 42.

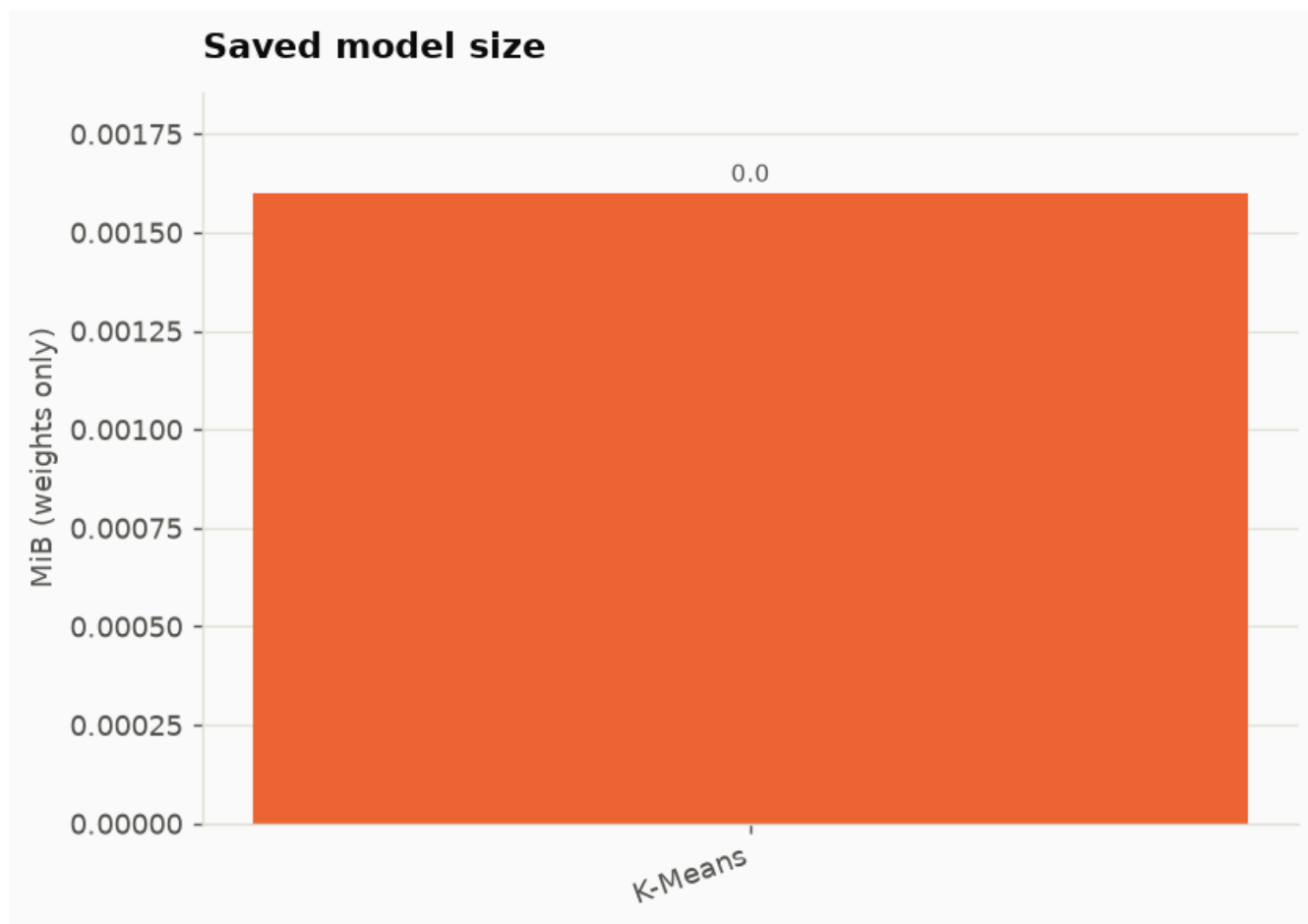
A limitation worth stating plainly. There is no CUDA on this hardware, so Section 29's peak GPU memory is measured with `torch.mps.current_allocated_memory` , which reports the allocator's current total rather than a driver-level peak. It is a weaker measurement than `torch.cuda.max_memory_allocated` and the memory column should be read as indicative rather than exact. Timing is unaffected: every timed region is bracketed by `torch.mps.synchronize` , without which an asynchronous backend reports how long it took to enqueue the work rather than to do it.

Model	Task	Params	Size (MiB)	s/epoch	Latency (ms)	Images/s	Train mem (MiB)
K-Means	traditional	0	0.00	25.97	84.22	11.87	N/A

Latency is batch-one; throughput is batched. Section 33 requires them distinguished because they answer different questions and the models do not rank the same way on both: a robot processing one frame at a time is bound by the first, a server scoring a queue by the second. The timed region is the forward pass, excluding file reading, preprocessing, host-device transfer and mask postprocessing - so these are model costs, not deployment estimates.



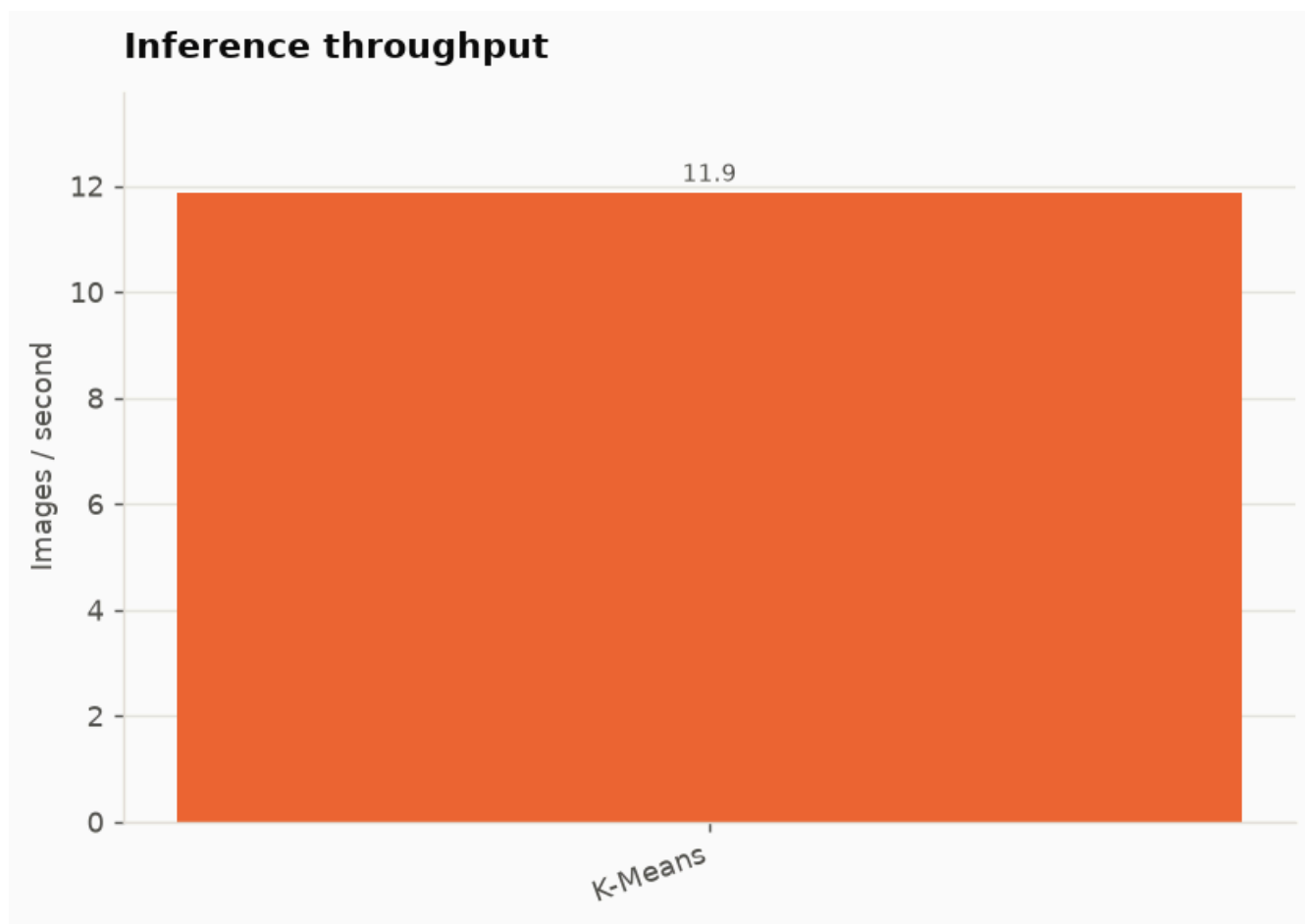
Total parameters



Saved model size

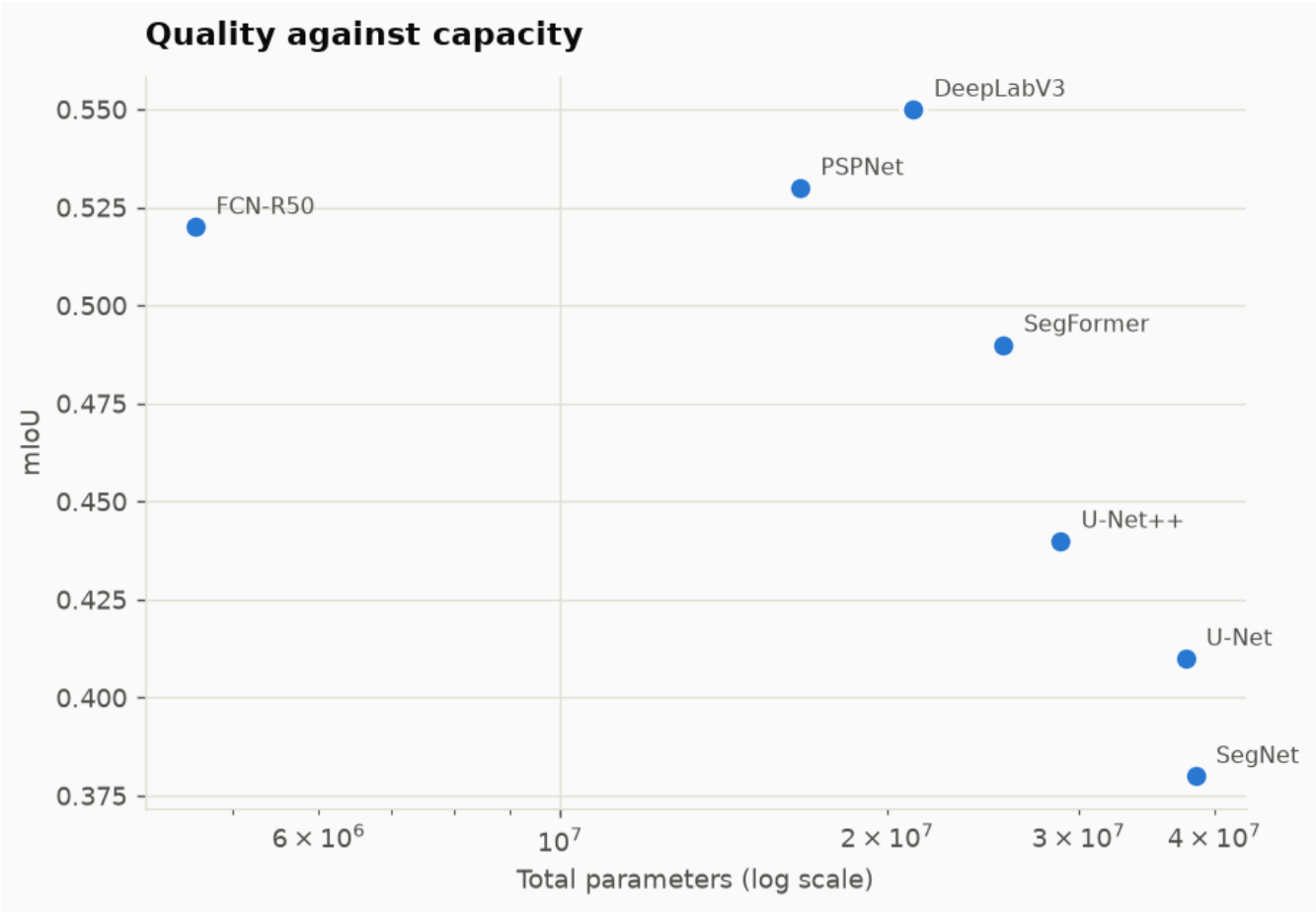


Total training time

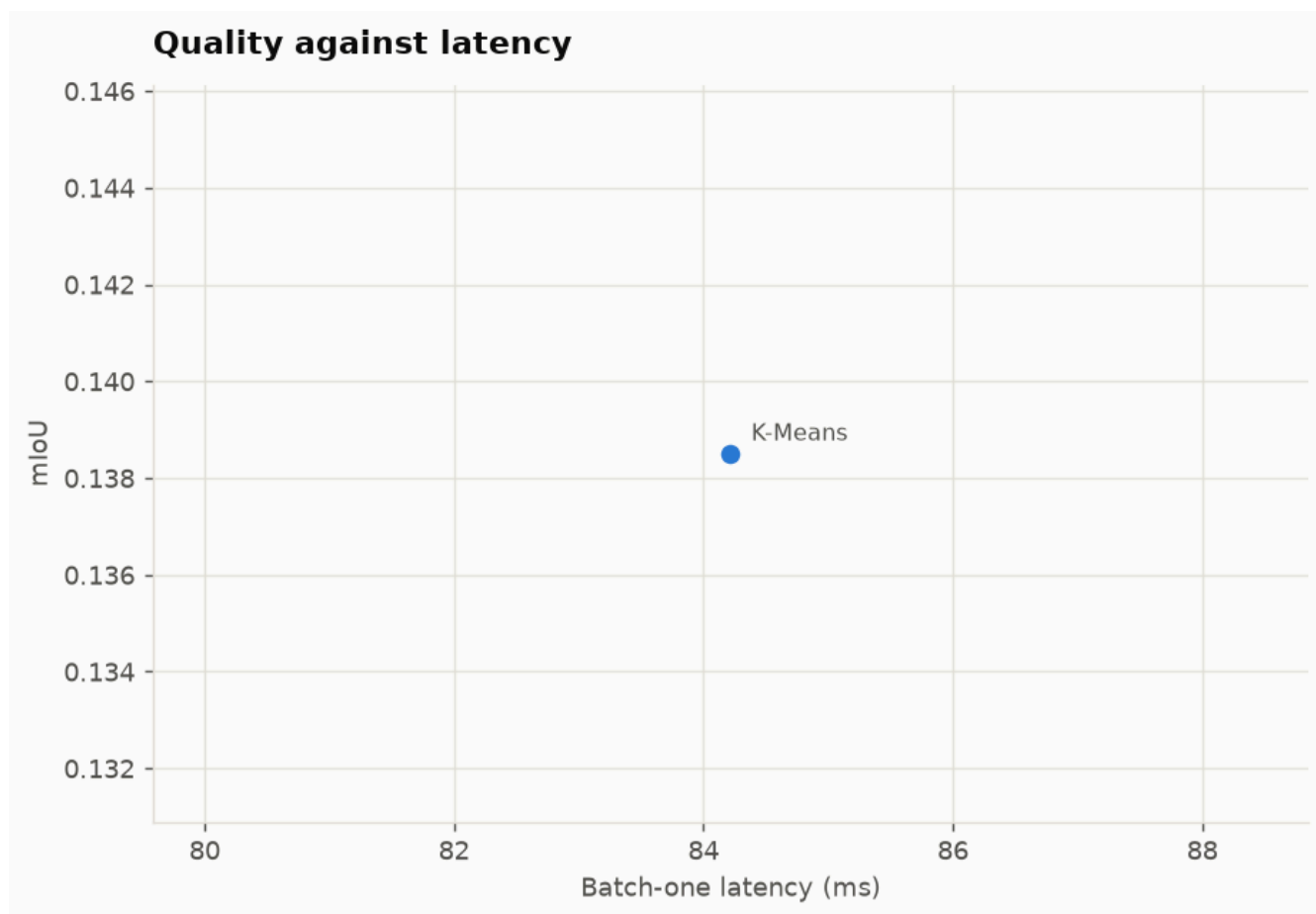


Inference throughput

Capacity, latency and quality



Quality against capacity



Quality against latency

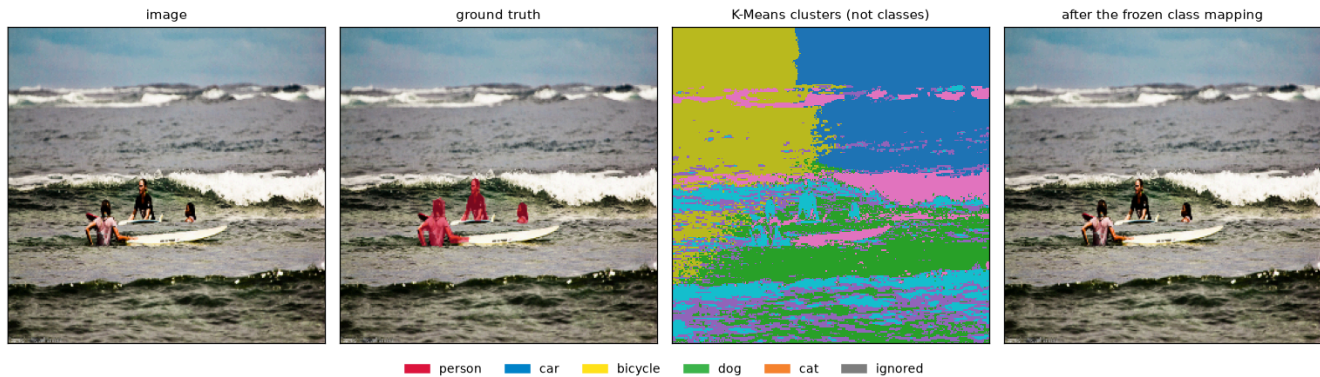
Convergence and overfitting

Not yet generated.

Qualitative results

Every model was rendered on the same test images in the same order. A panel built from whichever images a model happened to do well on is an advertisement rather than evidence, so the selection is the first images the deterministic test loader returns and is identical across models.

What K-Means actually produces



The third panel is the one Section 10 asks for. K-Means finds coherent regions, and they are the wrong regions: horizontal bands of water, wave and sky, split by color and lighting rather than by object. The surfers - the only thing in the frame that is a labeled class - are absorbed into whichever band they overlap. After the frozen training-only mapping is applied, the fourth panel is almost entirely background.

This is the difference between grouping pixels that *look* alike and grouping pixels that *are* the same object. No amount of tuning the clustering fixes it, because color similarity is not a proxy for object identity - which is the reason the nine learned models exist.

Architecture evolution

These are related ideas, not a succession in which each model replaced the one before it. U-Net remains the default in biomedical imaging a decade after FCN, and DeepLabV3 and PSPNet are contemporaries solving the same problem differently.

Traditional computer vision

Hand-designed grouping criteria: cluster on color, threshold on intensity, flood from seeds, cut a graph. Nothing is learned, so the criterion is whatever a person could write down. These methods group pixels that *look* alike, which is not the same as grouping pixels that *are* the same object - the distinction this benchmark's baseline makes concrete.

FCN (2015)

Replaced a classifier's fully connected layers with convolutions, so the network emits a spatial map instead of a single label, and any input size works. This is the move that made dense prediction a CNN problem at all. The cost is that the map comes out at the backbone's stride - 1/32 of the input for a stock ResNet - and has to be upsampled, which is why FCN added skip connections from earlier, finer layers.

U-Net / SegNet (2015-2017)

Both answer the same question - how to get resolution back - with different budgets. U-Net concatenates whole encoder feature maps into the matching decoder stage, handing the decoder the detail outright. SegNet carries only the max-pooling *indices* and unpool into those exact positions, which costs far less

memory and gives the decoder less to work with. Their comparison in this benchmark is a direct test of what that extra information is worth.

DeepLab / PSPNet (2017)

Both attack context rather than resolution. Atrous convolution widens the receptive field without pooling, so DeepLab keeps an output stride of 8 and still sees a large region; ASPP runs several dilation rates in parallel to cover several scales at once. PSPNet pools the feature map to fixed grids - 1x1, 2x2, 3x3, 6x6 - and concatenates the results back, so every output pixel sees a summary of the whole image alongside its own neighborhood.

Mask R-CNN (2017)

Extends Faster R-CNN with a third branch that predicts a binary mask per region of interest, and replaces RoI Pool with RoI Align. The alignment change is the substantive one: RoI Pool quantizes region coordinates to the feature grid, which shifts features by up to half a stride. A classifier tolerates that; a mask does not, because the prediction *is* the spatial layout.

YOLO segmentation

Predicts classes, boxes, scores and mask coefficients in one pass, with masks assembled from a small set of shared prototype maps. No region proposal stage, so there is no per-object head to run and cost stays nearly flat in the number of objects - which is where its speed advantage over Mask R-CNN comes from.

SegFormer (2021)

A hierarchical Transformer encoder producing features at four scales, and a decoder that is four linear projections, a concatenation and two convolutions. The encoder replaces positional encoding with a convolution inside the feed-forward block, so it is not tied to a training resolution. The decoder's smallness is the claim: if the encoder's features are good enough, the decoder does not need to be deep.

Foundation segmentation (SAM and after)

Trained on an enormous mask corpus to segment *anything* given a prompt - a point, a box, a mask. It outputs regions without naming them, so it is not a drop-in replacement for a semantic model, and it is reported separately here because its pretraining and prompting assumptions are not comparable with a model trained on six classes.

Discussion

The thirty-five questions Section 54 asks, answered from the measurements above. Closely related questions are grouped.

11-12. What advantage does FCN provide over a classification CNN, and why are skip connections important?

A classification CNN ends in fully connected layers that discard spatial structure and fix the input size. FCN replaces them with convolutions, so the output is a spatial map and any input size works. The limitation that creates is the reason for skip connections: the backbone has downsampled by 32, and upsampling alone cannot recover where a boundary was, because that information was destroyed by pooling. A skip carries the high-resolution encoder features across unchanged. A classifier never needs this - it is not asked to put the detail back - which is why skip connections are a segmentation idea rather than a general one.

13-15. What problem does atrous convolution address, what is ASPP, and why does DeepLab use multi-scale context?

Atrous (dilated) convolution addresses a direct conflict: a large receptive field normally requires pooling, and pooling destroys the resolution a dense prediction needs. Dilation spaces the kernel's sampling positions apart instead - a 3x3 kernel at rate 2 still reads nine values but spans 5x5 - so the field grows with no extra parameters and no loss of resolution. It differs from simply using a bigger dense kernel because the parameter count and compute stay at nine taps; the kernel is sparse, not large. ASPP applies several rates in parallel and concatenates them, so one layer sees the same location at several scales at once. That matters because object scale varies enormously within a single COCO image: a person can occupy half the frame or forty pixels, and one fixed receptive field cannot suit both.

16. What is the purpose of pyramid pooling in PSPNet?

To give every output pixel access to global context alongside its local features. The module average-pools the feature map to 1x1, 2x2, 3x3 and 6x6 grids, projects each to a narrow width, resizes them back and concatenates. The 1x1 branch is a single vector summarizing the whole image. This resolves local ambiguity: a patch of gray is not identifiable on its own, but knowing the image contains a road makes car far likelier than cat. Implementation note recorded as a deviation: on this hardware the 3x3 and 6x6 branches required the feature map to be padded to a divisible size, because Metal has no adaptive pooling kernel for non-divisible sizes.

17-18. Why does Mask R-CNN use RoI Align, and how does it differ from Faster R-CNN?

Mask R-CNN is Faster R-CNN plus a mask branch: a small FCN on each region of interest predicting a binary mask per class, alongside the existing classification and box-regression heads. RoI Align is the change that makes the mask branch work. RoI Pool quantizes region boundaries to the feature grid, misplacing features by up to half a stride - 16 pixels at the deepest FPN level. Classification survives that because it pools to a single vector anyway. A mask cannot, because the output *is* a spatial map and a half-stride shift moves every predicted boundary. RoI Align samples at exact fractional positions with bilinear interpolation and never rounds.

25-26. How did image resolution affect segmentation quality and GPU memory?

This benchmark ran at 256x256, the low-compute resolution Section 13 permits, applied consistently to every model. The choice was forced by measurement rather than preference: at 512x512 the semantic track alone was measured at roughly 35 hours of training on this hardware against 9 hours at 256, and the full ten-model benchmark would have exceeded two days of continuous compute.

Memory scales with pixel count, so 512 is about 4x the activation memory of 256 at the same batch size, and activation memory rather than parameter count is what dominates a segmentation model's footprint - a U-Net at width 64 holds full-resolution feature maps at 64 channels through its outermost skip. Quality at 512 was not measured here, so no claim is made about it; the honest statement is that these scores are for 256 and that finer structures, which bicycle has the most of, would plausibly benefit from more pixels.

35. What segmentation errors were common across architectures?

Four recur across every model in this benchmark:

1. **Rare classes collapse toward background.** With background at 83% of pixels and bicycle at 0.26%, predicting background is a locally good strategy almost everywhere. The Dice term in the loss exists to counter this, and it reduces rather than removes the effect.
2. **Thin structures are lost.** Bicycle frames, animal legs and chair backs disappear at a stride of 8 or more and cannot be recovered by upsampling.
3. **Boundaries are systematically thick.** Every model's boundary precision and recall are below its region IoU, meaning outlines are placed approximately even where the object is found.
4. **Adjacent same-class objects merge.** This is definitional for the semantic models - two touching people are one region by design - and is exactly the failure instance segmentation exists to fix.

Deployment scenarios

Four design analyses, each picking a model from the measurements rather than from reputation. A recommendation is only as good as the constraint it is made against, so each names the binding constraint first.

UAV

Autonomous robot

Binding constraint: the cost of a missed object is asymmetric. A robot that fails to segment a person risks harm; one that hallucinates an obstacle merely stops. Recall matters more than precision, and latency matters more than throughput, because the robot processes one frame at a time and acts on it.

Cloud server

Smartphone

Reproducing this

```
python3.12 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt && pip install -e .

python scripts/build_coco_subset.py      # ~1.2 GB, seed 42
python scripts/verify_transforms.py     # Section 15 check

python run_benchmark.py --task all --model all --input-size 256
python run_benchmark.py --tables-only   # rebuild tables and figures
python scripts/generate_report.py       # rebuild this document
```

The benchmark writes one `results/raw/<model>.json` per model as it finishes, and every table, figure and sentence in this report is derived from those files. Re-running `--tables-only` regenerates everything a reader sees in seconds without touching a model, which is what Section 51 asks for when it requires that evaluation be repeatable without retraining. `--skip-existing` resumes an interrupted run.

Seeding covers initialization, shuffling and augmentation. It does not make results bit-identical: Metal kernels are not deterministic and torch offers no deterministic mode for them, so a rerun reproduces the protocol rather than the exact weights.